DAY 3 | POST-READ

OOP and OpenAI Codex: Recap and Beyond

What we covered today, plus today's self-study topics: SOLID, decorators and generators.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
class and object	A class is a blueprint; objects are instances. __init__ sets up; self is the object.
Inheritance	class Child(Parent). super() calls the parent. Override to change behaviour.
Exceptions	try / except / else / finally. Catch specific errors. raise for your own rules.
Modules	Any .py file can be imported. A package is a folder of modules.
Dependency hygiene (NEW v4)	One venv per project, pinned requirements, pip-audit.
Codex workflow (NEW v4)	Describe, plan, small steps, test. Review every diff.
AGENTS.md (NEW v4)	Commands, style, folders not to touch, security rules.
Models and effort (NEW v4)	Light for routine, default for features, top tier only with approval.
Security and cost (NEW v4)	Scan AI code, never paste secrets, check daily token usage.

Part B: Self-study topics

1. SOLID in plain English

Letter	Principle	In one line
S	Single responsibility	One class, one job
O	Open / closed	Extend with new code, do not edit old code
L	Liskov substitution	Child classes can stand in for parents
I	Interface segregation	Small focused interfaces, not giant ones

Letter	Principle	In one line
D	Dependency inversion	Depend on abstractions, not concrete classes

2. Decorators: wrap a function

A decorator takes a function and returns a new one that adds behaviour, such as logging or timing. FastAPI routes on Day 5 are decorators. Use functools.wraps to keep the original name.

```
from functools import wraps
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b
```

3. Generators: values on demand

yield returns one value and pauses. The next call resumes. Generators use almost no memory, which is ideal for reading big log files line by line.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

print(list(countdown(3)))          # [3, 2, 1]
squares = (i * i for i in range(10**6))
print(next(squares), next(squares))   # 0 1
```

Part C: Going beyond today

1. Dataclasses

@dataclass writes __init__, __repr__ and __eq__ for you. Pydantic models (Day 5) follow the same idea with validation added.

2. Lockfiles and hashes

Tools such as pip-tools or uv produce lockfiles with exact versions and hashes for every dependency, including indirect ones. You will meet npm lockfiles on Day 9.

3. Layered AGENTS.md

Large repos can have an AGENTS.md in a sub-folder with extra rules for that folder. Codex reads the nearest one as well as the root file. Keep each one short and specific.

Part D: Check yourself

- What is the difference between a class and an object?
- Why should you avoid a bare except:?
- Why pin dependency versions, and what does pip-audit check?
- Name the four steps of structured AI-assisted coding.
- Give two things you must never paste into a Codex prompt.

Part E: Practice ideas for tonight

- Model a Shape base class with Circle and Rectangle subclasses that each implement area().
- Write a @timer decorator and a Fibonacci generator.
- Write an AGENTS.md for your SmartDesk folder using shared/templates/AGENTS_learner_template.md.

Coming up on Day 4: client server architecture, charts with Matplotlib, and storing SmartDesk tickets in MongoDB.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today